

Feedforward Neural Networks

Phuong T. Nguyen



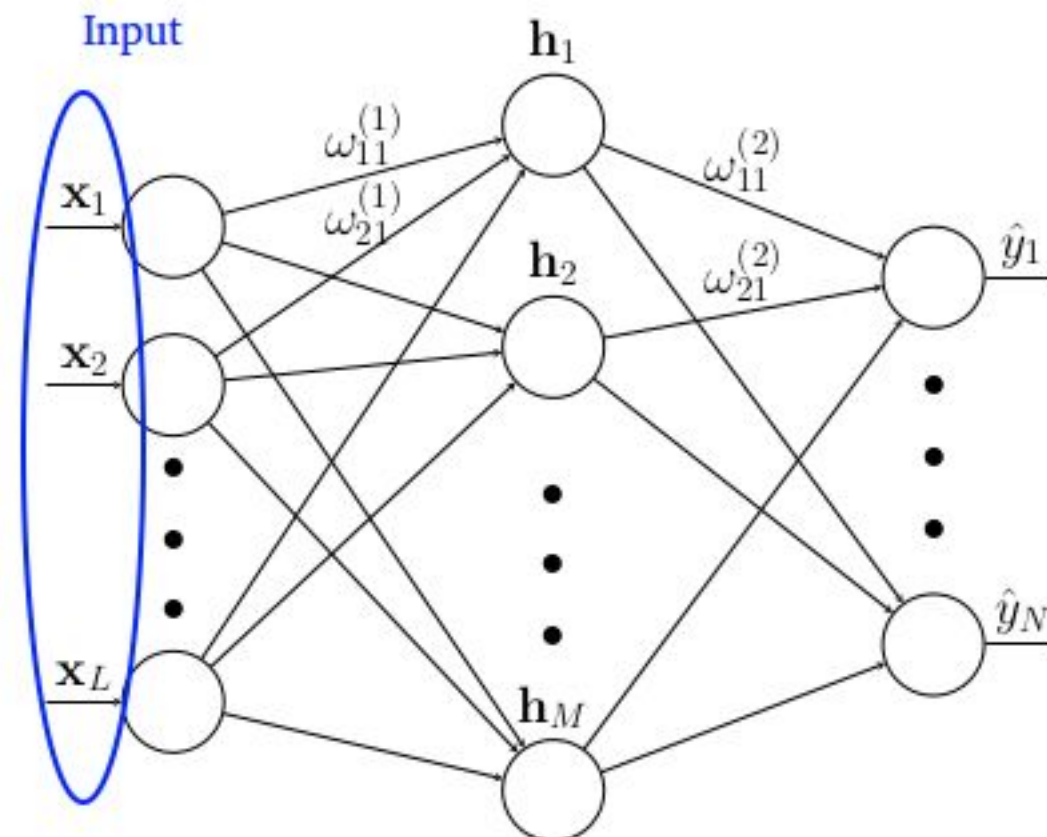
Dipartimento di Ingegneria e Scienze
dell'Informazione e Matematica

Università degli Studi dell'Aquila

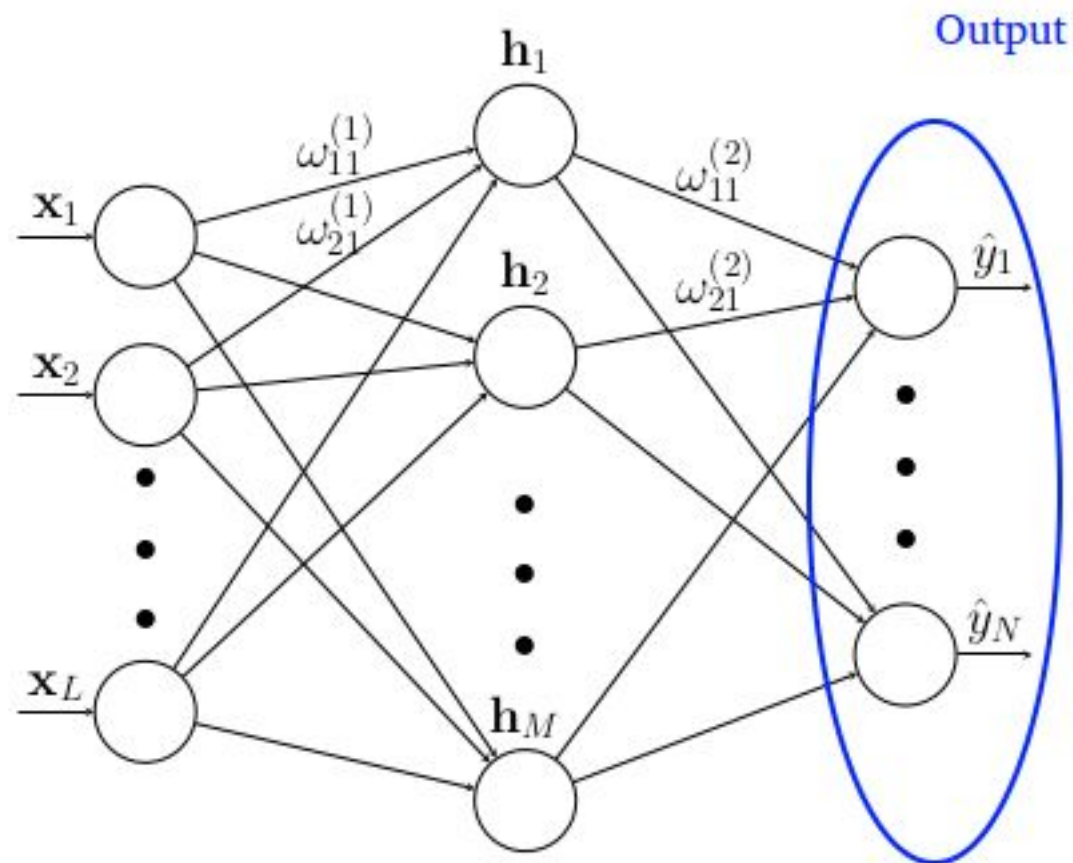
Introduction

- A Feed-Forward Neural Network (FFNN) is one of the most basic types of artificial neural networks used in machine learning and deep learning.
- It is called feed-forward because the information flows in one direction: from input to output, without any cycles or loops in the network.
- The network consists of layers of neurons, where each layer feeds its output to the next layer.
- It is commonly used for tasks like classification, regression, and pattern recognition.

Network configuration



Network configuration (2)



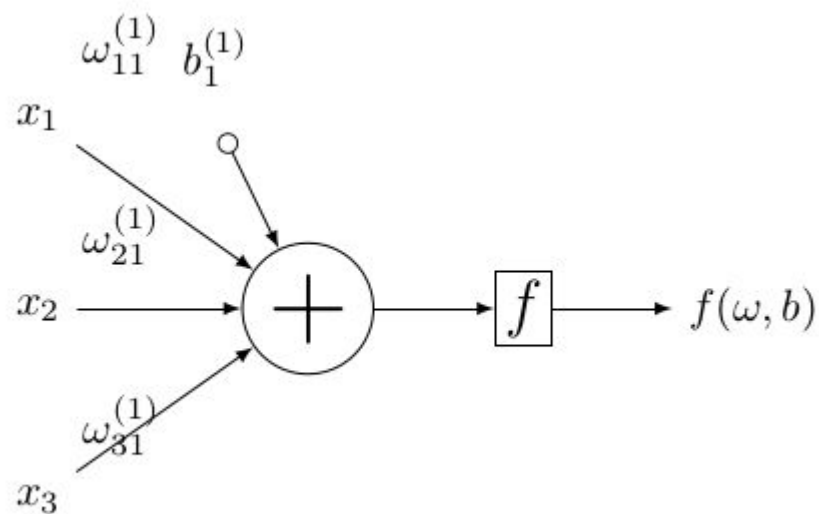
Network configuration (3)

- **Input Layer:** This is the first layer, where data enters the network. The number of neurons in the input layer corresponds to the number of features in the dataset.
- **Hidden Layers:** These are the intermediate layers where the actual computations take place. FFNNs can have one or more hidden layers.
- Each neuron in a hidden layer takes inputs from the previous layer, processes them, and passes them forward.

Output layer

- This is the final layer, which produces the network's output.
- The number of neurons in the output layer depends on the task, i.e., in a binary classification problem, the output layer might have one neuron, while in a multi-class classification problem, it might have one neuron per class.

Neural Network: Perceptron



(a) A simple perceptron with 3 inputs

$$output = f\left(\sum_{i=1}^3 \omega_{i1}^{(1)} \cdot x_i + b^{(1)}\right)$$

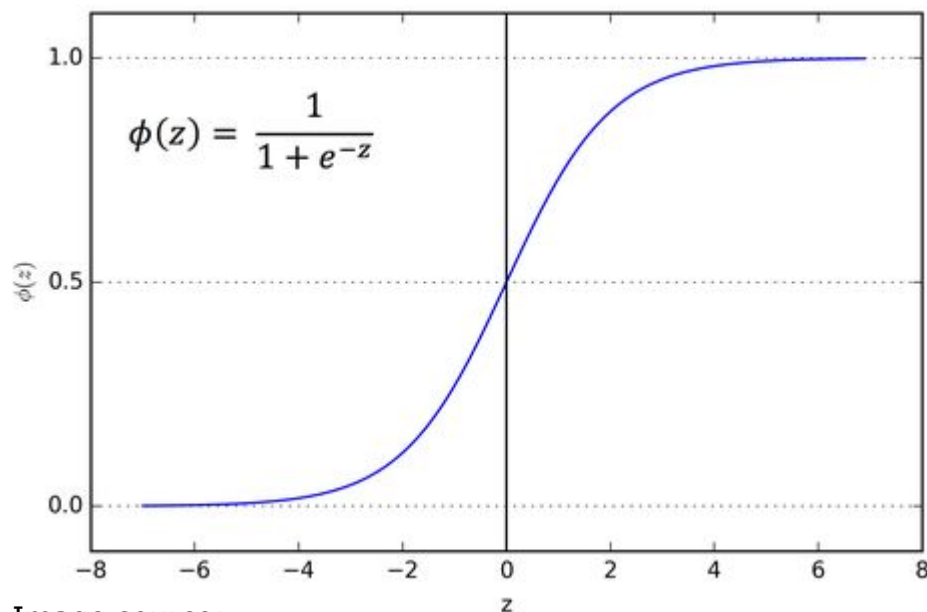
- A decision making unit, taking into consideration different inputs.

Neurons and Weights

Each neuron in a feed-forward neural network performs the following operations:

- **Weighted Sum:** The neuron receives input values, multiplies each input by a weight, and computes the sum.
- **Activation Function:** The weighted sum is passed through an activation function to introduce non-linearity into the model, allowing the network to solve more complex problems.

Activation

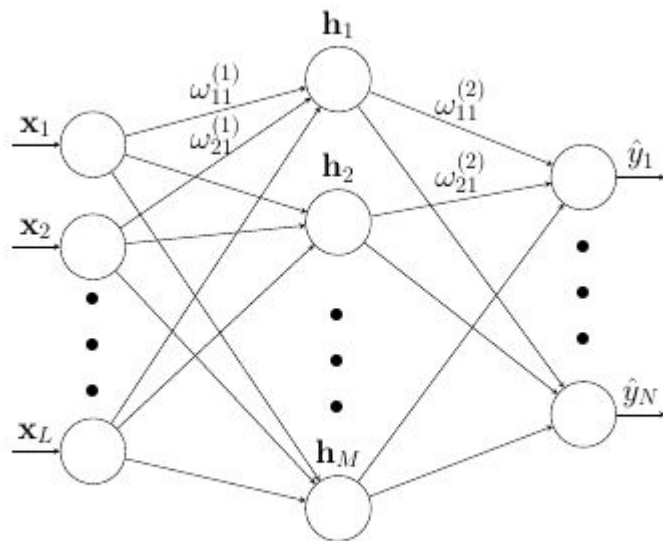


$$y = f(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-\sum_j w_j x_j - b)}$$

Image source:
<https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6>

- Small changes in the weights (w) and biases (b) will trigger a small change in the output.

Learning



(b) A three-layer neural network

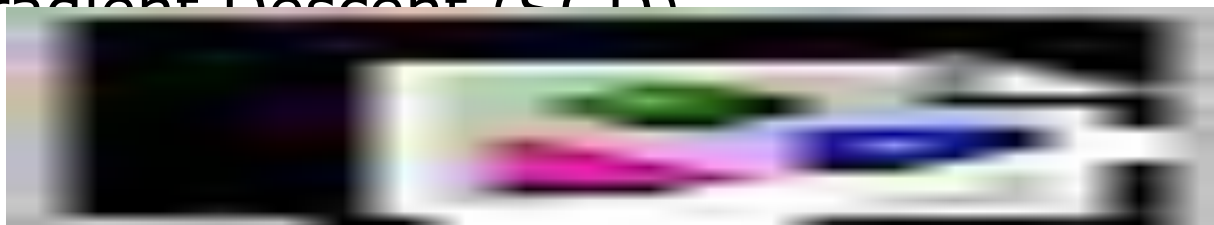
$$C(w, b) = \frac{1}{2n} \sum_x \|y(x) - (w.x + b)\|^2$$

$$\Delta y \approx \sum_j \frac{\partial y}{\partial w_j} \Delta w_j + \frac{\partial y}{\partial b} \Delta b$$

- Learning is actually the process of modifying the weights so that we can produce the desired outputs given the inputs.
- The more (good) input data you feed the network, the better accuracy you will obtain.

Learning

- Find a set of **weights and biases** (w, b) that minimize the cost function between the **predicted values** and the **real ones** using Stochastic Gradient Descent (SGD)



- Δy is a linear function of Δw and Δb

$$\Delta y \approx \sum_j \frac{\partial y}{\partial w_j} \Delta w_j + \frac{\partial y}{\partial b} \Delta b$$

Training

Training a feed-forward neural network involves adjusting the weights and biases of the neurons to minimize the loss function. The most common training method is backpropagation combined with gradient descent:

- Backpropagation computes the gradients of the loss function with respect to each weight in the network by propagating the error backward from the output layer to the input layer.
- Using the gradients computed by backpropagation, the weights are updated to minimize the loss. The learning rate controls the size of the weight updates.

Derivative and gradient descent

- Derivative is used to find the position where a function (with a few variables) reaches the minimum.
- When the function becomes complicated with many variables, and the change can happen with every variable, then we need gradient descent.
- Gradient descent is used to training data to iteratively update/improve a machine learning model.

■ Training (2)

- During the forward pass, input data is fed through the neural network layer by layer, with each layer performing a weighted sum of inputs followed by an activation function.
- This process continues until the output layer is reached, and the final predictions are generated.

Loss Function

- FFNN uses a loss function (or cost function) to quantify the error between the predicted output and the true label. Common loss functions include:
 - Mean Squared Error (MSE) for regression tasks.
 - Cross-Entropy Loss for classification tasks.

Loss calculation

- After obtaining the network's predictions, the loss (error) between these predictions and the actual target values is computed using a loss function.
- The loss function measures how far off the predictions are from the true values

$$C(w, b) = \frac{1}{2n} \sum_x \|y(x) - (w.x + b)\|^2$$

Gradient descent

- It plays a central role in adjusting the network's weights and biases to minimize the error or loss function during the training process.
- The primary goal of training a feedforward neural network is to find the optimal set of weights and biases that minimize the difference between the predicted outputs and the actual target values for a given dataset.
- A loss function (cost function) is defined to quantify the error or discrepancy between predictions and targets.

Gradient descent (2)

- Gradient descent is an iterative optimization algorithm used to minimize the loss function.
- It works by updating the network's weights and biases in the opposite direction of the gradient of the loss function with respect to these parameters.
- The gradient represents the slope or rate of change of the loss function, indicating the direction in which the parameters should be adjusted to reduce the error.

Back Propagation

- The backpropagation algorithm is used to compute the gradient of the loss function.
- Since the cost is a function of outputs from the network, to understand how the cost varies with earlier weights and biases, it is necessary to repeatedly apply the chain rule, working backward through the layers to obtain usable expressions.

Back Propagation (2)

- Backpropagation calculates the gradients of the loss with respect to each weight and bias by propagating errors backward through the network.
- It involves computing the gradients layer by layer, starting from the output layer and moving backward through the hidden layers.

Back Propagation (3)

- The gradients of the loss with respect to each parameter (weights and biases) are computed.
- Starting from the output layer, the gradient of the loss with respect to the output layer's activations is also computed.
- This gradient represents how much a small change in the output activations affects the loss.
- The specific form of the gradient depends on the choice of loss function.

Back Propagation (4)

- Once the output layer gradients are computed, the gradients are propagated backward through the network.
- The gradient at each layer is computed based on the gradient at the following layer.
- This is done using the ***chain rule of calculus***. The gradient at a hidden layer is the product of the gradient at the next layer and the derivative of the activation function at that layer.

Minimizing the cost function

- Find a set of weights that minimize the cost function between the predicted value and the real one using gradient descent.

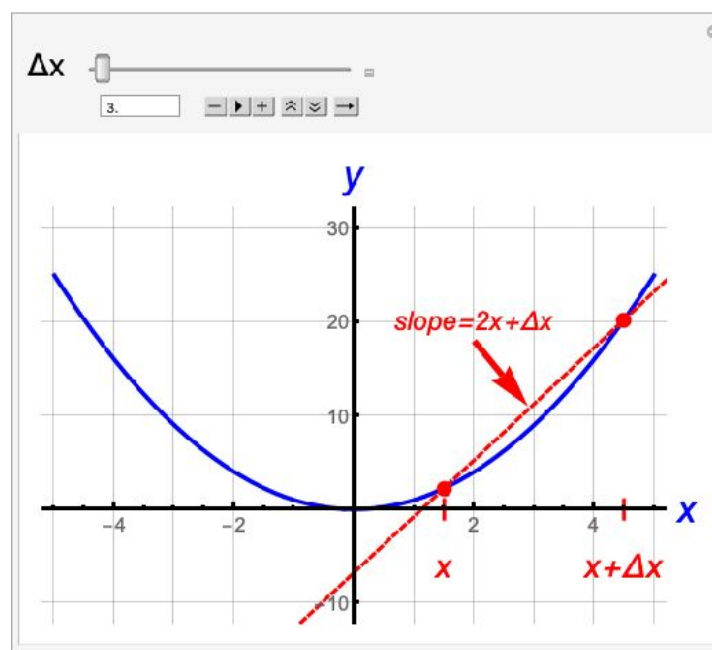
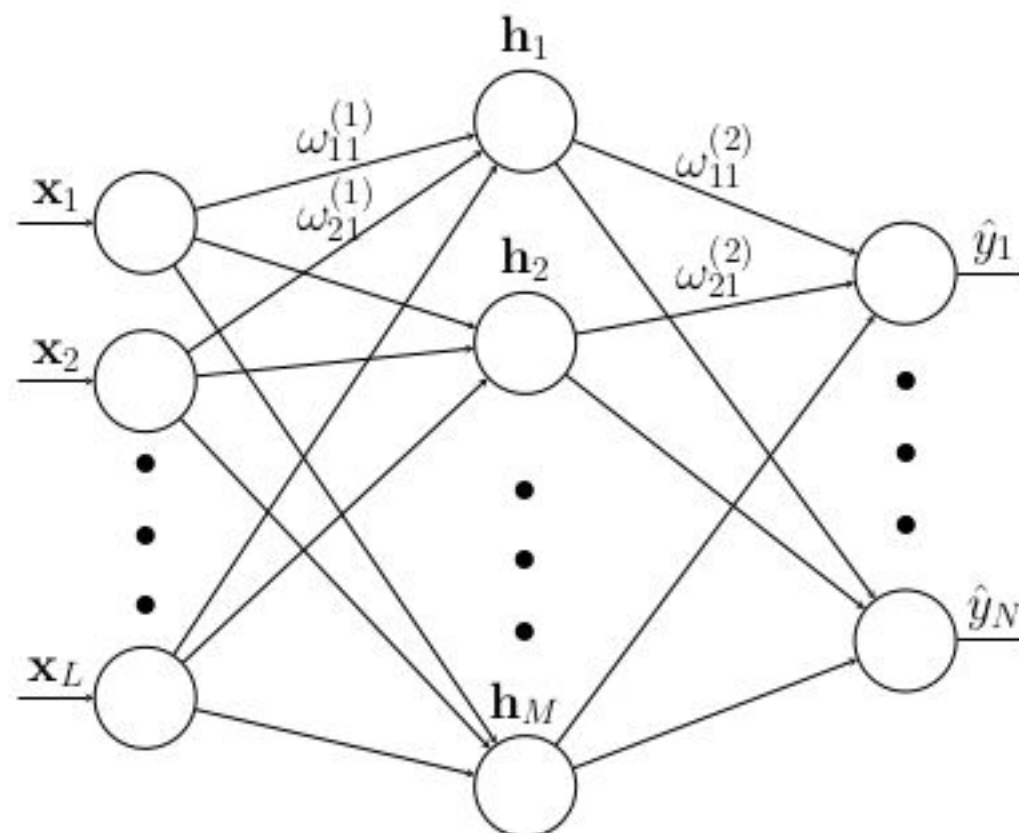


Image source: <https://infinityisreallybig.com/2019/09/20/the-derivative-of-y-x2/>

Network configuration



L1 Regularization

- L1 regularization, also known as Lasso (Least Absolute Shrinkage and Selection Operator), adds a penalty equivalent to the absolute value of the magnitude of the coefficients (weights) to the loss function or L1 regularization, the modified loss function becomes:

$$\text{Loss} = \text{Original Loss} + \lambda \sum_{i=1}^n |w_i|$$

- λ is a hyperparameter that controls the strength of the regularization (how much penalty is applied).
- w_i are the model's weights.

Effect of L1

- L1 regularization encourages the model to learn sparse weights, i.e., it drives some of the coefficients (weights) to exactly zero.
- As a result, it performs feature selection by removing irrelevant features, simplifying the model.
- If many features are not useful, L1 regularization can shrink their weights to zero, effectively eliminating them from the model.

L2 Regularization

- L2 regularization, also known as Ridge regression, adds a penalty equivalent to the square of the magnitude of the coefficients (weights) to the loss function. The modified loss function becomes:

$$\text{Loss} = \text{Original Loss} + \lambda \sum_{i=1}^n w_i^2$$

- λ is a hyperparameter that controls the strength of the regularization (how much penalty is applied).
- w_i are the model's weights.

Effect of L2

- L2 regularization encourages the model to distribute the error across all the weights by shrinking them but not necessarily to zero.
- L2 discourages large weights, leading to more stable models.
- Unlike L1, L2 does not result in sparse weights (weights becoming zero). Instead, it reduces the size of all coefficients more evenly.

L1 and L2

- L1 or L2 regularization is applied to the weights of the neurons in each layer of the network (except the input layer).
- This means that the penalty is imposed on the weights connecting one layer to the next.
- Regularization is not applied to the biases of the neurons, as biases are usually not penalized in practice.

Softmax function

- The last layer of a neural network is often normalized using the softmax function, especially in classification tasks where the goal is to produce probability distributions over multiple classes.
- The softmax function is used to transform the raw output of the network into class probabilities.
- The raw output of the neural network's last layer, often referred to as logits, represents unnormalized scores for each class (can be positive, negative, or zero).

Softmax function

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Image source: <https://deepai.org/machine-learning-glossary-and-terms/softmax-layer>

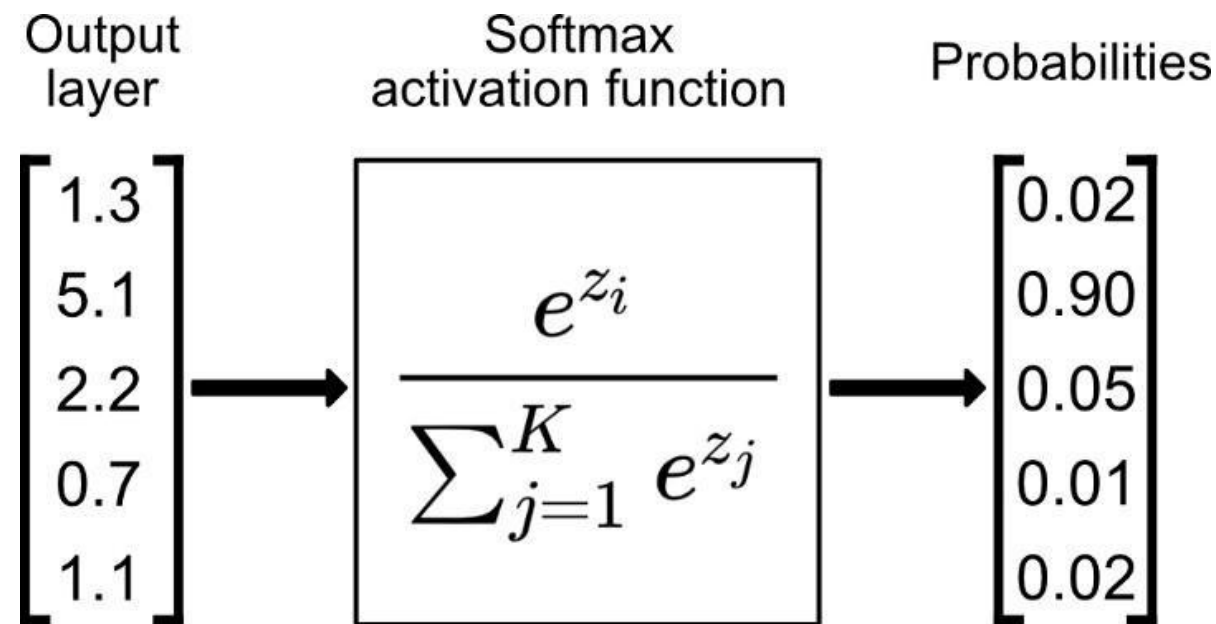


Image source: <https://towardsdatascience.com/softmax-activation-function-explained-a7e1bc3ad60>

- Used in the last layer of a neural network to normalize the output.
- Converts K real values into a vector of K real values that sum to 1.

Softmax layer

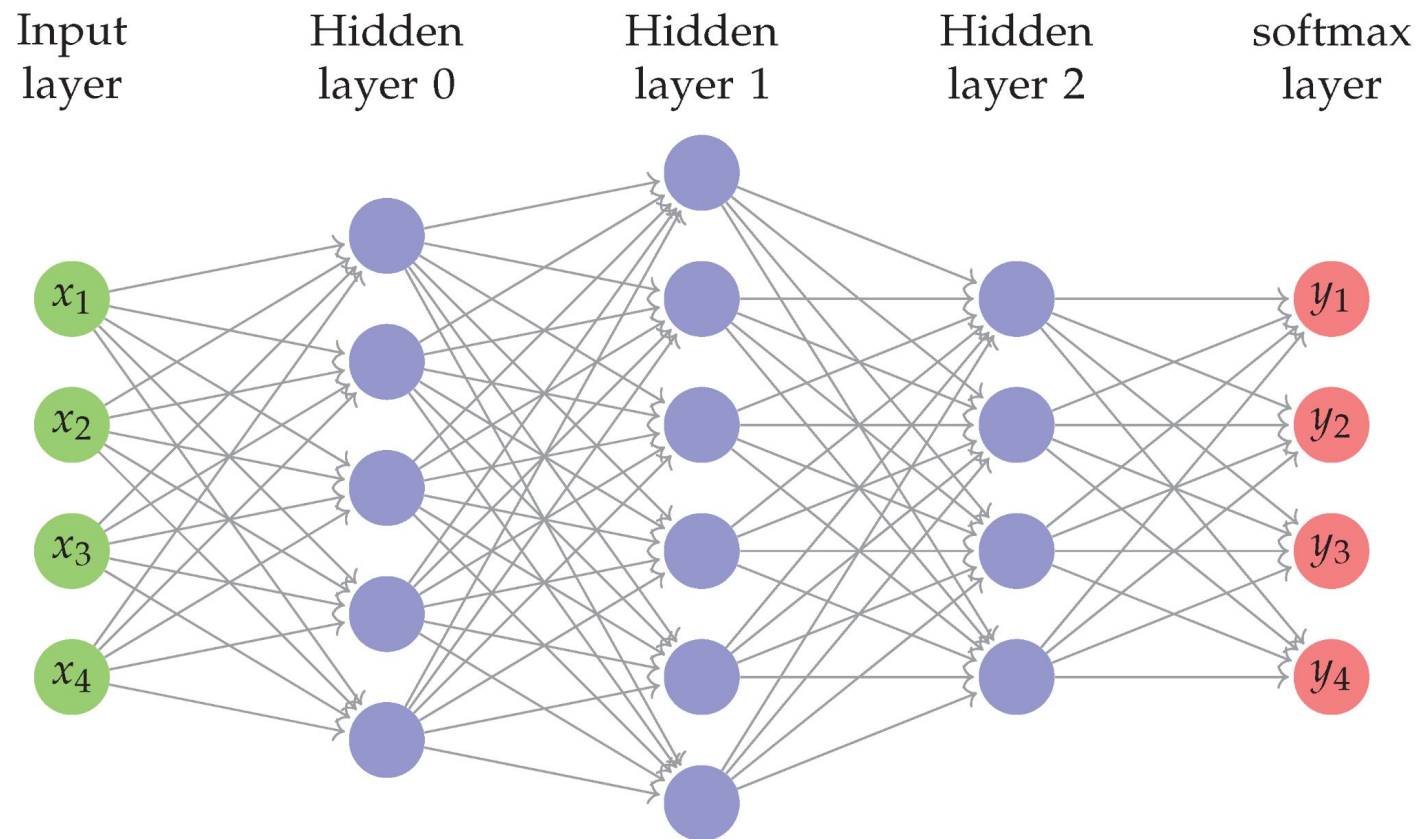
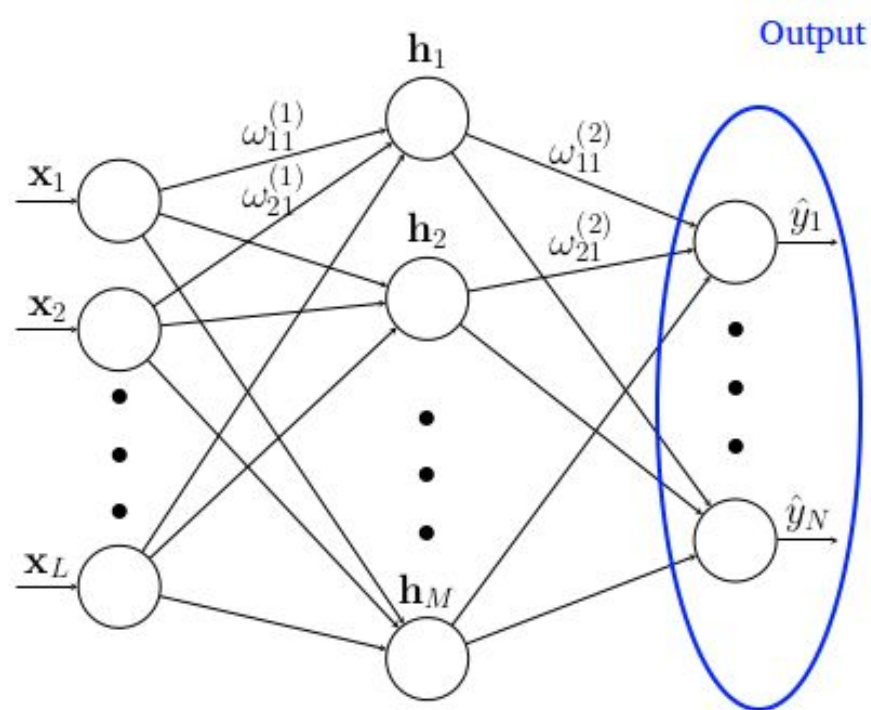
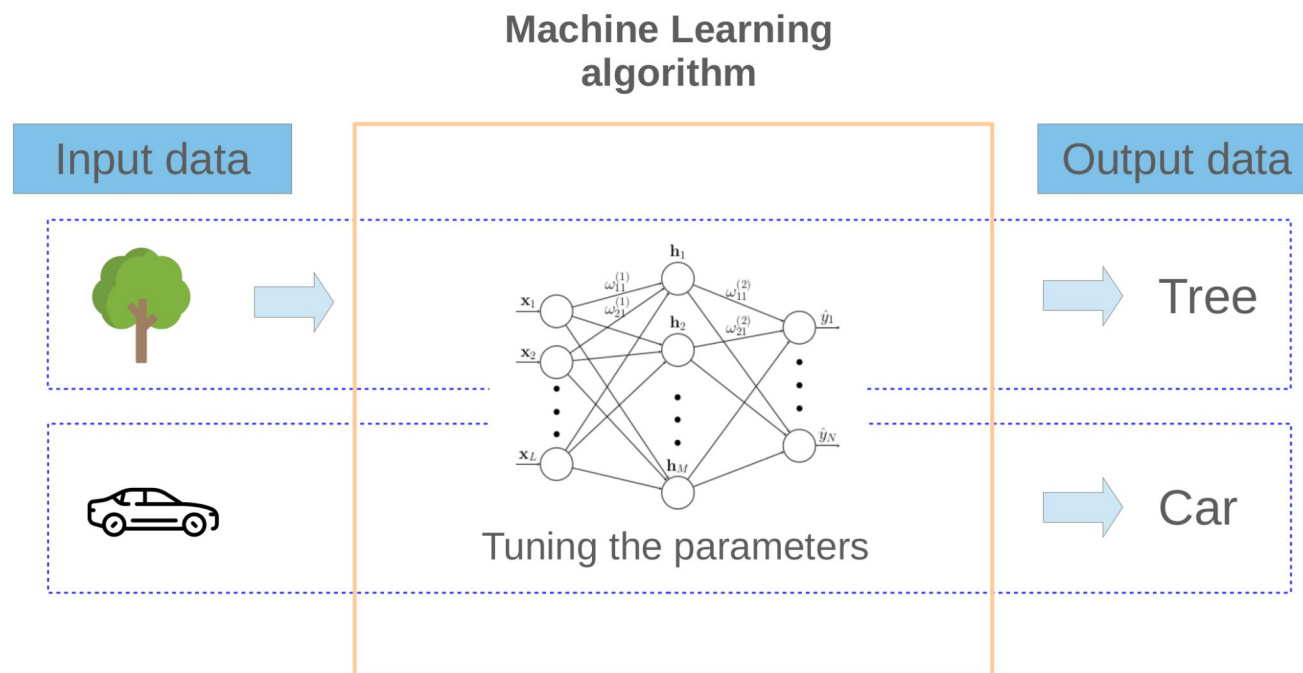


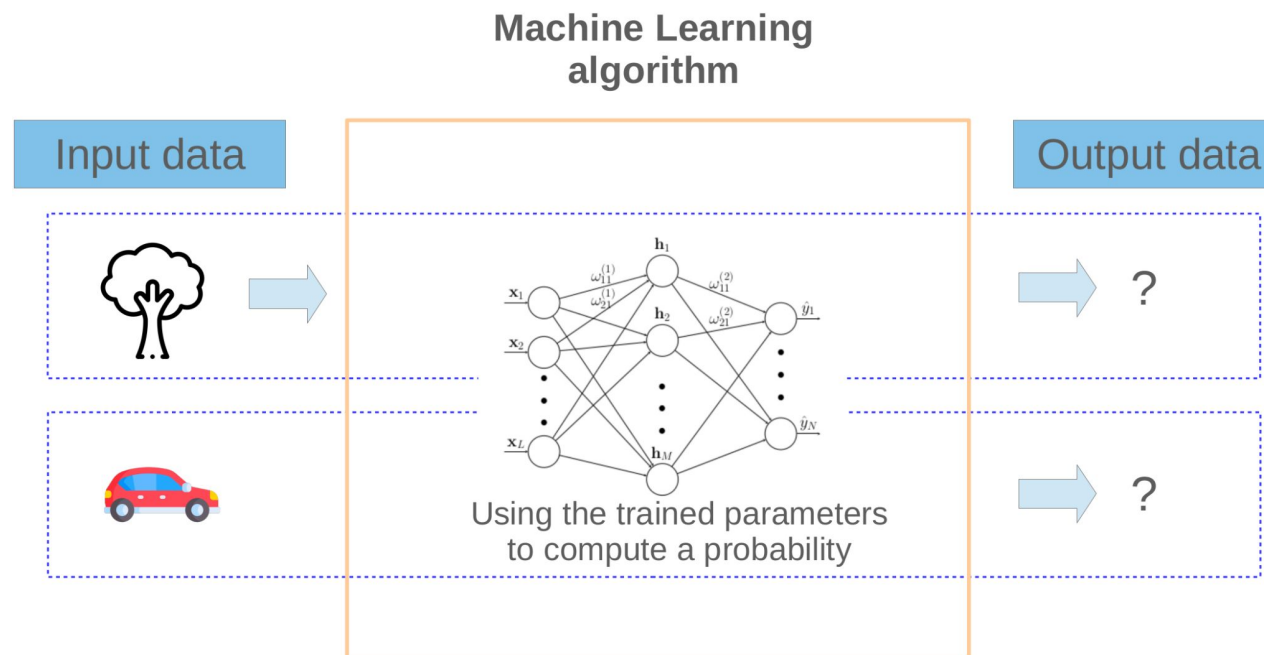
Image source: <https://www.mdpi.com/2227-7080/8/3/46/htm>

Training phase



- Feature: Input data (documents, images).
- Label: Given by humans.

Testing phase



- Feature: Input data (documents, images).
- Label: The algorithm tries to compute the probability, and give a label.

Recognition of handwritten characters

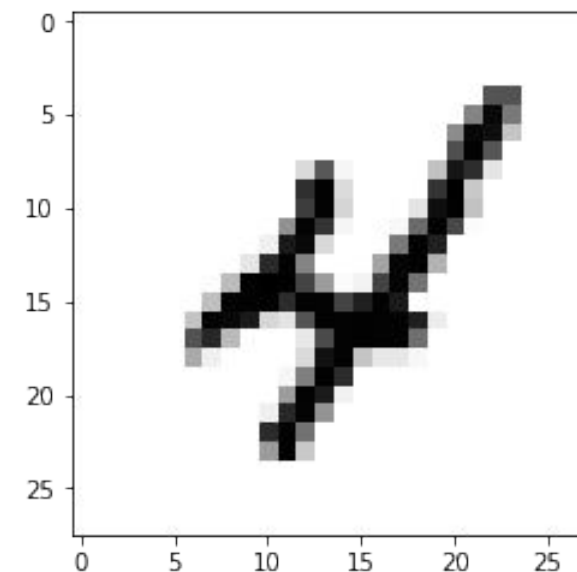
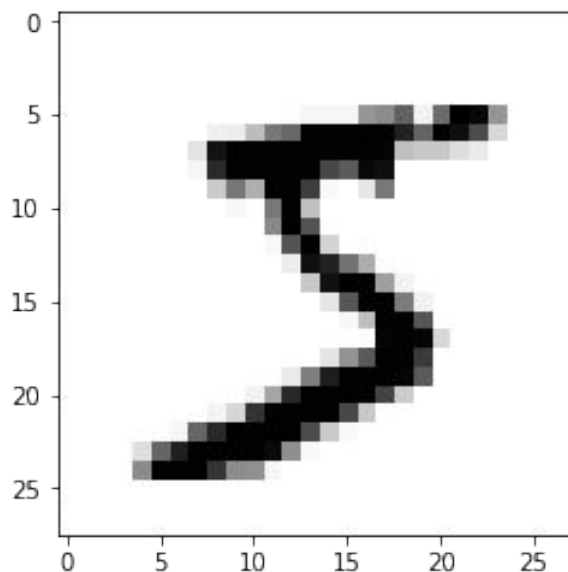
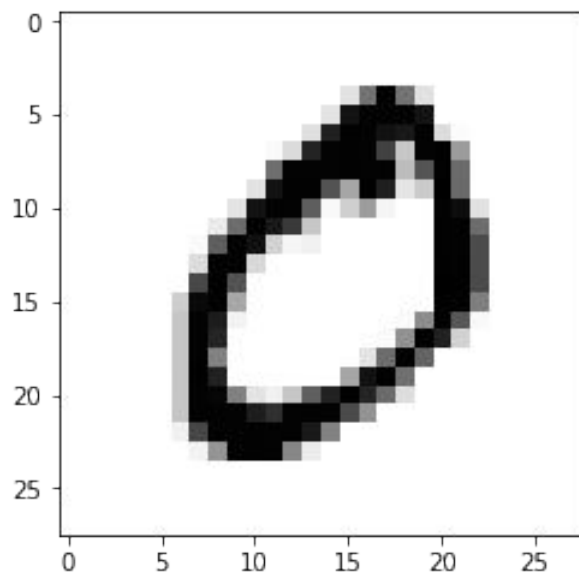


Image source: https://www.python-course.eu/neural_network_mnist.php

- Image is a 28 x 28 pixel matrix.
- Each cell represents a pixel whose value ranges from 0 to 255.

A number represented in grayscale values



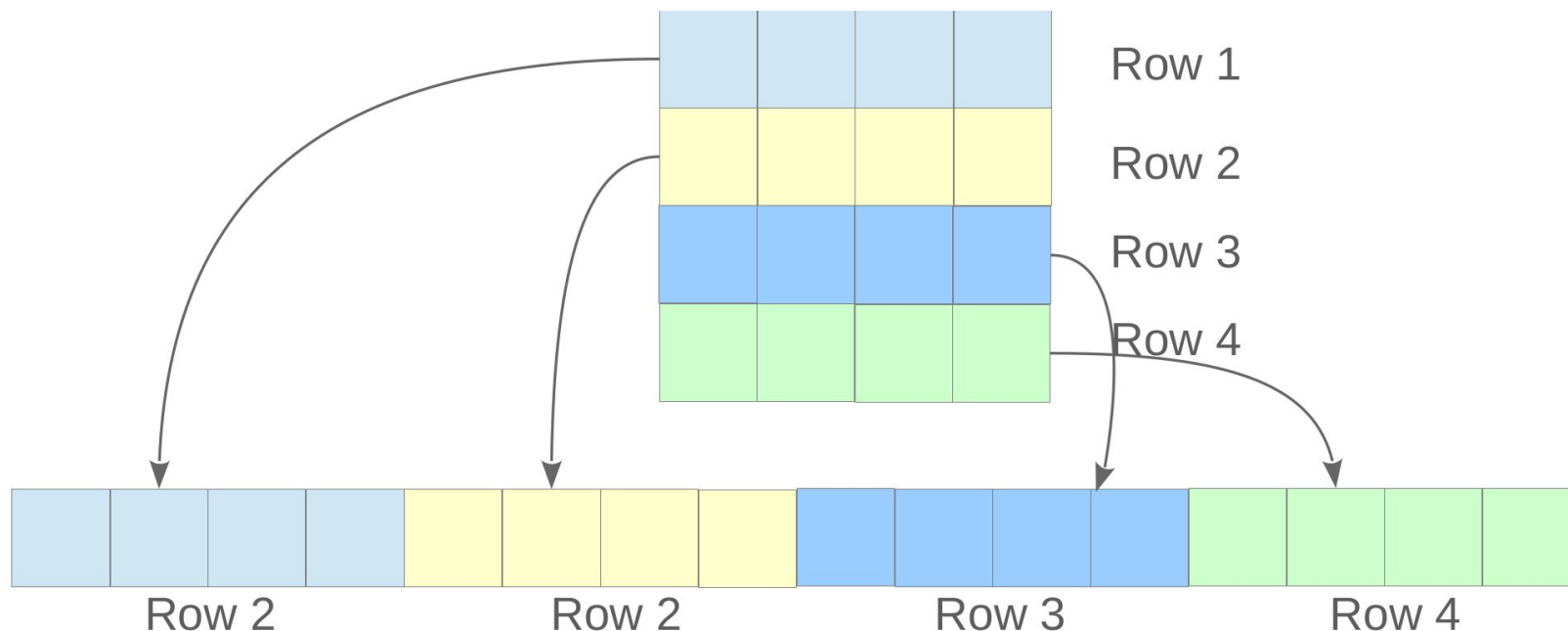
0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

Image source: <https://insightsimaging.springeropen.com/articles/10.1007/s13244-018-0639-9>

- A grayscale value of 0 represents a black point.
- A grayscale value of 255 represents a white point.
- The values in between correspond to different shades of gray.

Flattening the images



- Each image is flattened as an array with 784 elements containing grayscale level of one pixel (from 0-255).

Spam Detection

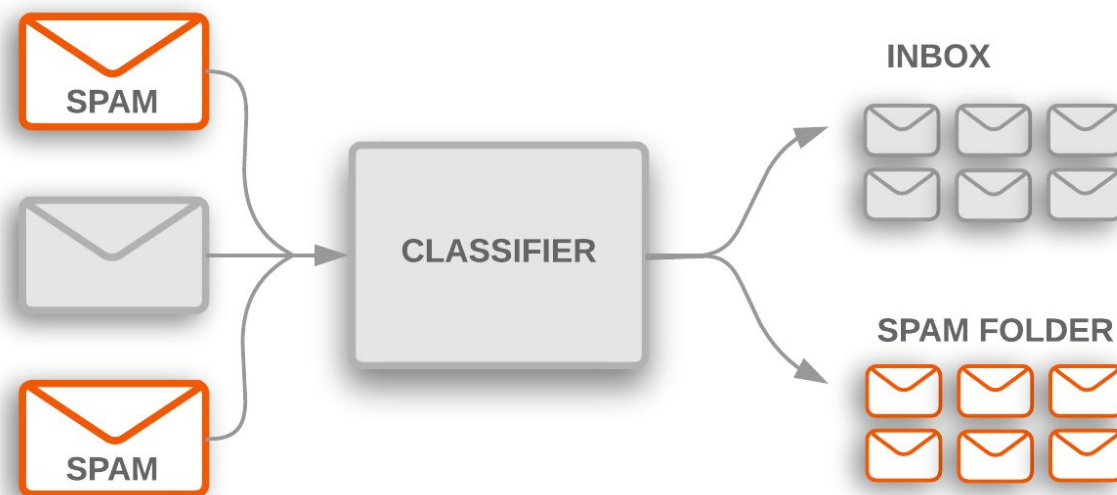


Image source: <https://developers.google.com/machine-learning/guides/text-classification>

- Features: the text within email.
- Training data: "spam" and "not spam" manually classified by humans.
- Deployment: an email is fed into the network, which returns the desired label of the email.

Spam Detection: Data Processing

Document Vectorization

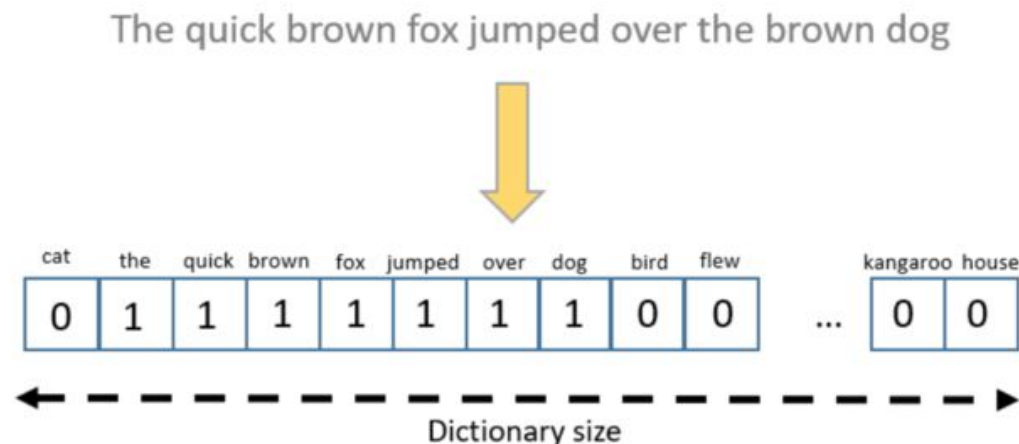


Image source:
<https://www.kdnuggets.com/2019/11/text-encoding-review.html>

- Features: the text within emails is parsed using a suitable format.
- Remove punctuation and stop words, lemmatization.
- Vectorize the data using bag of words (BOW).

Document vectorization

COUNT VECTORIZATION



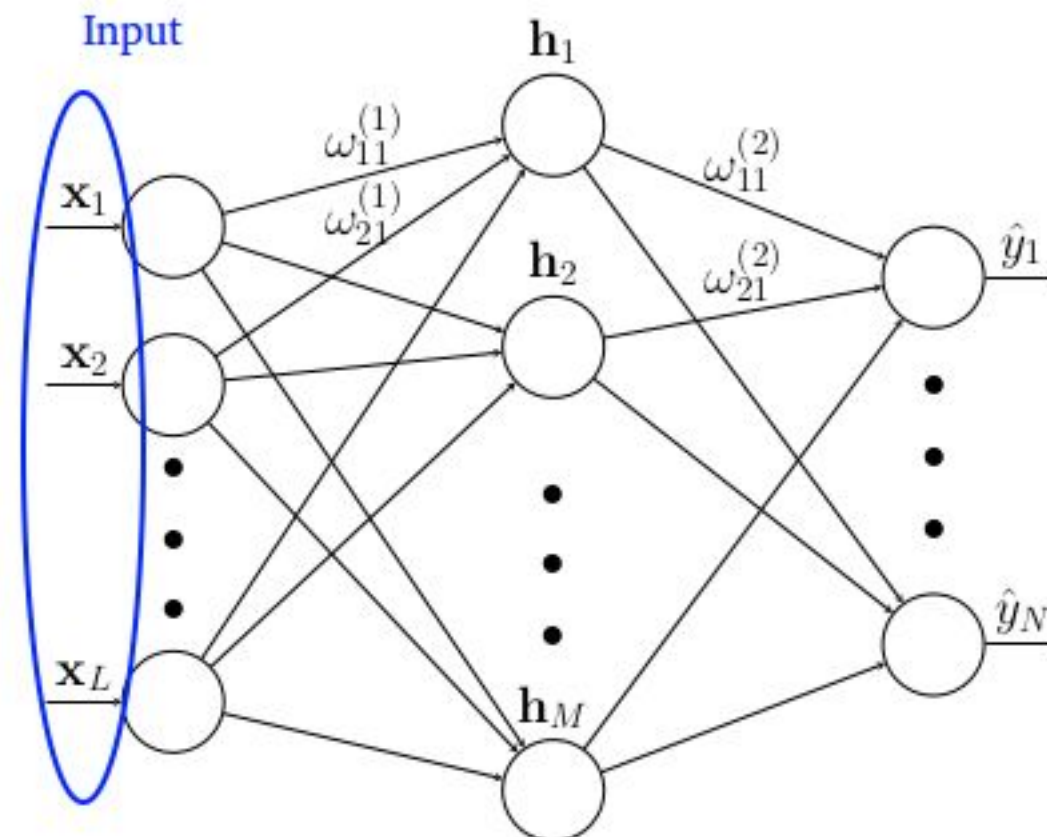
	text
0	Eddard Stark is a king in the north.
1	A king but one king : kings are everywhere.
2	Hodor was different : he was not a king .
3	But the North could not change without him.

	king	was	the	not	But	him	one	north	kings	is	in	he	Eddard	everywhere	different	could	change	but	are	Stark	North	Hodor	without
0	1	0	1	0	0	0	0	1	0	1	1	0	1	0	0	0	0	0	0	1	0	0	0
1	2	0	0	0	0	0	1	0	1	0	0	0	0	1	0	0	0	1	1	0	0	0	0
2	1	2	0	1	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	1	0
3	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	0	1

Image source: <https://knowledge.dataiku.com/latest/courses/nlp-visual/text-handling/text-handling-summary.html>

- Count the frequency of occurrence.
- Form the document vector, each entry corresponds to one term/word.

Feeding input data



Hyperparameters: Epoch and batch

- A sample/instance is a row of data.
- Batch size: a number of samples passed through the model before updating the weights.
- After one batch, the predicted labels and the real ones are compared to compute the error.
- Epoch: the number of times that the entire training data is used to train the model.

Learning rate

- The learning rate is a hyperparameter that determines the step size of each weight and bias update during gradient descent.
- It controls the balance between convergence speed and stability.
- Choosing an appropriate learning rate is crucial; too large a learning rate can lead to overshooting the minimum, while too small a learning rate may result in slow convergence.

Mini batch

- In practice, gradient descent is often performed on mini-batches of data rather than the entire dataset.
- Mini-batch gradient descent strikes a balance between computational efficiency and convergence speed.
- The gradients are computed for a subset (mini-batch) of the training data and used to update the parameters.

■ Mini batch (2)

- In its extreme form, mini-batch gradient descent with a mini-batch size of 1 is known as stochastic gradient descent (SGD).
- SGD updates the parameters after processing each individual training sample.
- SGD introduces randomness, which can allow the model to escape local minima, however this may result in noisy updates.

■ Mini batch (2)

- To prevent overfitting, regularization techniques like L1 and L2 regularization are often used in conjunction with gradient descent.
- Early stopping is another strategy to prevent training for too long and overfitting the data.

Advantages

- FFNNs are easy to implement and serve as the foundation for more complex neural networks.
- They can be applied to a wide range of tasks like classification, regression, and even pattern recognition.
- Theoretically, a feed-forward neural network with enough neurons and layers can approximate any continuous function, making it very powerful for modeling complex relationships.

Disadvantages

- FFNNs need large amounts of data to generalize well, especially as the number of layers increases.
- Since information only flows forward, FFNNs do not have memory, thus it is not suitable for tasks involving sequential data (like time series or language processing).
- When FFNNs become deeper (more hidden layers), training can become slower, especially if the network suffers from the vanishing gradient problem during backpropagation.

— Take-away messages

- It is the backbone of training feedforward neural networks, iteratively updating the model's parameters by computing gradients using backpropagation, moving towards the optimal set of weights and biases that minimize the loss function
- Proper tuning of hyperparameters, such as the learning rate, is crucial for successful training and convergence.